Day 14 — SQL Fundamentals

Topics: Introduction to SQL, CREATE/ALTER/DROP, INSERT/UPDATE/DELETE, SELECT, WHERE/ORDER BY/LIMIT, filtering, constraints, primary and foreign keys.

Examples use MySQL syntax. They also work in PostgreSQL and SQLite with small changes (e.g. AUTO_INCREMENT becomes SERIAL or AUTOINCREMENT).

1. Create the database and tables

```
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE courses (
    course_id   INT PRIMARY KEY AUTO_INCREMENT,
    course_name VARCHAR(100) NOT NULL UNIQUE,
    credits     INT NOT NULL CHECK (credits > 0)
);

CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    name       VARCHAR(100) NOT NULL,
    email      VARCHAR(100) UNIQUE,
    age        INT CHECK (age >= 16),
    city       VARCHAR(50) DEFAULT 'Unknown',
    course_id  INT,
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

Constraints used:

- **PRIMARY KEY** makes each row unique and non-null.
- **FOREIGN KEY** links students.course_id to courses.course_id.
- **NOT NULL, UNIQUE, CHECK, DEFAULT** validate the data.

ALTER and DROP:

```
ALTER TABLE students ADD phone VARCHAR(15);       -- add a column
ALTER TABLE students MODIFY phone VARCHAR(20);    -- change its type
ALTER TABLE students DROP COLUMN phone;           -- remove it

-- DROP TABLE students;       -- deletes the table
-- DROP DATABASE college_db;  -- deletes everything
```

2. Insert student and course records

Insert courses first, because students refer to them.

```
INSERT INTO courses (course_name, credits) VALUES
('Python Programming', 4),
('Data Science', 5),
('Web Development', 3),
('Machine Learning', 5);

INSERT INTO students (name, email, age, city, course_id) VALUES
('Aarav Sharma',   'aarav@mail.com',  20, 'Pune',       1),
('Priya Patil',    'priya@mail.com',  22, 'Mumbai',     2),
('Rohan Deshmukh', 'rohan@mail.com',  19, 'Aurangabad', 1),
('Sneha Kulkarni', 'sneha@mail.com',  21, 'Nagpur',     3),
('Vikram Singh',   'vikram@mail.com', 23, 'Pune',       4),
('Anjali Joshi',   'anjali@mail.com', 20, 'Aurangabad', 2);
```

3. Retrieve and filter records

```
SELECT * FROM students;                              -- all rows
```

```
SELECT name, city FROM students;                    -- specific columns

SELECT * FROM students WHERE city = 'Pune';         -- equality
SELECT * FROM students WHERE age > 20;              -- comparison
SELECT * FROM students WHERE age BETWEEN 19 AND 21;  -- range
SELECT * FROM students WHERE city IN ('Pune', 'Mumbai');
SELECT * FROM students WHERE name LIKE 'A%';        -- starts with A
SELECT * FROM students WHERE city = 'Pune' AND age > 20;
SELECT * FROM students WHERE city = 'Nagpur' OR age < 20;
SELECT * FROM students WHERE email IS NOT NULL;

SELECT * FROM students ORDER BY age DESC;           -- sort
SELECT * FROM students ORDER BY city ASC, name ASC;
SELECT * FROM students ORDER BY age DESC LIMIT 3;   -- top 3 oldest

-- Bonus: JOIN to show each student's course name
SELECT s.name, c.course_name
FROM students s
JOIN courses c ON s.course_id = c.course_id;
```

4. Update student information

```
UPDATE students SET city = 'Mumbai' WHERE name = 'Rohan Deshmukh';
UPDATE students SET age = age + 1 WHERE city = 'Pune';
UPDATE students SET email = 'priya.patil@mail.com', age = 23
WHERE student_id = 2;
```

Note: Always include WHERE. Without it, every row is updated.

5. Delete selected records

```
DELETE FROM students WHERE student_id = 6;
DELETE FROM students WHERE city = 'Nagpur' AND age < 22;
```

Deleting a course that students still use fails because of the foreign key:

```
DELETE FROM courses WHERE course_id = 1;  -- Error: foreign key constraint
```

You must delete or reassign those students first. This is the constraint protecting your data.

6. Practice exercises

1. List students from Aurangabad.
2. Show the 2 youngest students.
3. Find students whose name contains "an".
4. Count students per course (hint for tomorrow: GROUP BY).
5. Add a grade column, then update it for one student.
6. Try inserting a student with a duplicate email. What error do you get?
7. Try inserting a student with course_id = 99. What happens, and why?

Expected output checklist

• SHOW TABLES; lists courses and students.
• DESCRIBE students; shows the keys and constraints.
• SELECT * FROM students; returns your inserted rows (fewer after DELETE).
• Constraint violations (duplicate email, invalid foreign key) are rejected.